

Relatório Individual de Produção

Projeto Pokedéx — Consolidado (Etapas 1 e 2)

Aluno: Luís Leão

Disciplina / Professor: Akira

Grupo: G10

Repositório do grupo (GitHub): <https://github.com/poo-ec-2026-1/g10>

Data: julho de 2026

1. Atribuição de cargo e tarefas

Cargo / responsabilidade atribuída:

Fui responsável pela camada de Banco de Dados do projeto ao longo das duas etapas. A camada compreende a conexão com o banco (classe Database), os repositórios que realizam as operações de persistência (padrão Repository) e a rotina de carga que popula o banco a partir dos arquivos CSV. A implementação utiliza ORMLite (mapeamento objeto-relacional) sobre SQLite, seguindo o padrão do tutorial da disciplina.

1.1 Etapa 1 — Atribuição inicial

Na Etapa 1, a atribuição foi estruturar a camada de banco a partir de um arquivo CSV com os 151 Pokémon da primeira geração. As tarefas incluíam:

- Tratar o arquivo CSV de origem (padronização de cabeçalho e codificação).
- Implementar a conexão com o banco (classe Database).
- Implementar os repositórios (padrão Repository) que dão acesso aos dados.
- Implementar a rotina de carga (PopularBanco) que lê os CSVs e popula o banco.
- Documentar a camada (LEIA-ME e relatório técnico).

1.2 Etapa 2 — Ampliação da atribuição

Na Etapa 2, a responsabilidade se ampliou para atender novas necessidades da interface. As tarefas adicionais foram:

- Manter e evoluir a camada entregue anteriormente.
- Implementar dois novos repositórios (EquipeRepository e MembroTimeRepository) solicitados pela interface para permitir a persistência do time montado pelo usuário.
- Padronizar esses novos repositórios seguindo o mesmo padrão dos anteriores.
- Ampliar a documentação inline (Javadoc), atendendo ao feedback recebido na Etapa 1.
- Preparar e gravar a apresentação em vídeo da minha parte do projeto.

O que foi exercido na prática:

Exerci todas as responsabilidades atribuídas nas duas etapas. Além disso, atuei como integrador entre a camada de banco de dados e as demandas da interface: quando o colega responsável pela interface enviou uma primeira versão dos repositórios em um padrão diferente do adotado no projeto (método salvar em vez de create, recebendo ConnectionSource em vez de Database), identifiquei a inconsistência e reescrevi ambos no padrão unificado do projeto, garantindo coerência arquitetural.

2. Contribuição de acordo com a atribuição

2.1 Etapa 1 — O que foi entregue

A camada de banco foi implementada seguindo três responsabilidades, conforme o padrão do tutorial: conexão (Database), repositórios (um por entidade, com operações CRUD e consultas) e carga de dados (PopularBanco, que lê os CSVs e popula o banco). Os repositórios operam sobre as classes de modelo (Pokemon, Golpe, Nature, TipoEfetividade), que pertencem à etapa de Modelagem do projeto.

Arquivos entregues:

Arquivo	Função
Database.java	Gerencia a conexão com o arquivo SQLite (abrir/fechar).
PokemonRepository.java	Operações sobre os Pokémon: create, loadFromId, loadAll, busca por nome, filtro por tipo e cálculo de fraquezas/vantagens.
GolpeRepository.java	Operações sobre o catálogo de golpes.
NatureRepository.java	Operações sobre as natures.
TipoEfetividadeRepository.java	Operações sobre a tabela de efetividade entre tipos (TypeChart).
PopularBanco.java	Rotina de carga: lê os CSVs e popula o banco via repositórios; gera o arquivo pokedex.db.
pokemons.csv	Dados-base: os 151 Pokémon (número, nome, tipos, atributos e imagem).
golpes.csv	Catálogo de golpes (nome, tipo, poder, precisão, categoria).
pokemon_golpes.csv	Associação entre cada Pokémon e o seu golpe característico.
LEIA-ME.md	Instruções de instalação das bibliotecas e de execução.

Tratamento dos dados de origem:

- Padronização do cabeçalho do CSV (colunas sem espaços nem acentos) e leitura em UTF-8, preservando caracteres especiais (por exemplo, Nidoran♀ e Nidoran♂).
- Tipo secundário ausente passa a ser nulo no banco, em vez de texto vazio.
- Associação de um golpe característico a cada Pokémon (uso visual, exibido no card).
- Inclusão de dados canônicos que não constavam no CSV: a tabela de efetividade entre os 18 tipos (apenas os confrontos não neutros) e as 25 natures.

Resultado da carga:

Conteúdo	Quantidade
Pokémon cadastrados	151
Golpes no catálogo	91
Natures	25
Confrontos na tabela de efetividade	120

A verificação confirmou que todos os Pokémon foram importados, que cada um possui exatamente um golpe associado e que não há violações de integridade referencial nos dados.

2.2 Etapa 2 — O que foi adicionado

- Implementação do `EquipeRepository`, seguindo o mesmo padrão: dao estático, `setDatabase` para preparar DAO e tabela, e operações `create`, `loadFromId`, `loadAll`, `update`, `delete` e `deleteById` (CRUD completo, já que um time é editável).
- Implementação do `MembroTimeRepository`, também no padrão `Repository`, com o CRUD completo e um método adicional `loadByEquipe(equipe)`, que utiliza `QueryBuilder` do `ORMLite` para retornar os membros de uma equipe específica — técnica nova em relação às consultas simples usadas na Etapa 1.
- Padronização dos dois novos repositórios em relação aos entregues na Etapa 1: mesmo estilo de construtor, mesma nomenclatura de métodos, mesma estrutura de tratamento de erros.
- Ampliação da documentação inline (Javadoc) dos métodos principais nos repositórios `PokemonRepository`, `EquipeRepository`, `MembroTimeRepository` e na classe `Database`, respondendo diretamente ao ponto "faltou documentação" apontado no feedback da Etapa 1.
- Manutenção da camada entregue anteriormente (`Database`, `PokemonRepository`, `GolpeRepository`, `NatureRepository`, `TipoEfetividadeRepository` e `PopularBanco`), sem regressões.
- Preparação e gravação da apresentação em vídeo da minha parte do projeto, com explicação dos arquivos e das linhas mais relevantes.

2.3 Três commits mais relevantes

- 1) <https://github.com/poo-ec-2026-1/g10/commit/b7353e85ee5f75d89a487e6c3b7515dbfb932ac7> — Adiciona documentação Javadoc no `PokemonRepository`, cobrindo os métodos `create`, `loadFromId`, `loadAll`, `loadByName`, `loadByType` e `fraquezasDe` (o mais complexo, que combina os tipos com a tabela de efetividade).
- 2) <https://github.com/poo-ec-2026-1/g10/commit/a870860118d0feda0a1e72b84362fe9c57359c4c> — Adiciona documentação Javadoc no `MembroTimeRepository`, descrevendo o CRUD completo e o uso do `QueryBuilder` no método `loadByEquipe`.
- 3) <https://github.com/poo-ec-2026-1/g10/commit/7e4bdf4882d82591049edfefd9ff1714f2616aa9> — Adiciona documentação Javadoc no `Database`, descrevendo o construtor, o `getConnection` (com reuso da conexão) e o `close`.

2.4 Resposta ao feedback da Etapa 1

Na Etapa 1 recebi retorno positivo sobre a implementação da carga de banco e o uso de `LIKE`, além do reconhecimento pela criação das estruturas de `Efetividade` e `Natures`. O ponto identificado como oportunidade de melhoria foi a documentação. Nesta Etapa 2, ataquei diretamente esse ponto: adicionei documentação Javadoc nos métodos principais dos repositórios e da classe `Database`, ampliei o `LEIA-ME` com uma seção sobre os novos repositórios, e passei a fazer commits com mensagens descritivas

2.5 Principais dificuldades

- Aprender o padrão ORM (`ORMLite`) a partir do tutorial da disciplina e aplicá-lo a uma base maior que a do exemplo, com múltiplas entidades relacionadas.

- Decidir como representar dados que não estavam no CSV (tabela de efetividade e natures) sem comprometer a simplicidade da camada.
- Padronizar os repositórios enviados por um colega, mantendo o padrão único do projeto sem retrabalho para os demais integrantes.
- Coordenar as fronteiras entre a minha camada e a etapa de Modelagem, para garantir que os repositórios operem sobre entidades compatíveis.

3. Contribuição além do atribuído

Além do que foi atribuído estritamente à camada de banco, contribuí com a equipe nos seguintes pontos ao longo das duas etapas:

- Tratamento do CSV original (Etapa 1): disponibilizei o arquivo em formato limpo (UTF-8, cabeçalho padronizado) para uso compartilhado por toda a equipe.
- LEIA-ME.md do banco (Etapa 1, ampliado na Etapa 2): descreve as bibliotecas necessárias e o passo a passo no BlueJ — material reaproveitável por quem for executar o projeto.
- Relatório técnico em PDF (Etapa 1): documentação da camada de banco de dados (arquitetura, decisões e resultados), disponível no repositório do grupo.
- Proposta de arquitetura em camadas (Etapa 1): defini com o grupo que as classes de modelo (entidades) ficam sob responsabilidade da etapa de Modelagem e que minha camada opera sobre elas, o que reduziu retrabalho e alinhou as fronteiras entre as partes do projeto.
- Padronização técnica (Etapa 2): identifiquei e corriji a inconsistência de padrão nos repositórios enviados por um colega, garantindo coerência arquitetural.
- Ponte técnica com a interface (Etapas 1 e 2): mantive comunicação direta com o responsável pela interface, ajustando os métodos dos repositórios ao que a interface efetivamente precisa consumir (loadAll, loadByName, loadByType, fraquezasDe, loadByEquipe).
- Apresentação em vídeo (Etapa 2): preparei roteiro e gravei a apresentação da minha parte com explicação linha a linha das partes mais relevantes do código.

Commits / documentos adicionais relevantes:

- <https://github.com/poo-ec-2026-1/g10/commit/3247b34773df46867870f4544bef306dfb6de132> — Commit-base da camada de banco (Etapa 1), sobre o qual esta etapa evoluiu, mantido como referência técnica.

4. Considerações gerais

4.1 O que aprendi

- Aplicação prática do padrão ORM (ORMLite) e do padrão Repository: substituir SQL escrito à mão por operações sobre objetos Java reduz a complexidade do código e melhora a separação de responsabilidades.
- Importância de tratar a fonte de dados (CSV) antes da carga: pequenas decisões como codificação UTF-8 e tratamento de campos vazios evitam problemas que só apareceriam mais tarde, na interface.
- Uso do QueryBuilder do ORMLite para consultas com filtro, além dos métodos simples (queryForId, queryForAll) usados na Etapa 1.
- Como manter um padrão arquitetural único quando o projeto cresce e novos componentes são adicionados por diferentes integrantes.
- Uso adequado de Git ao longo de um trabalho em grupo: commits pequenos, focados e com mensagens descritivas, em vez de uploads em bloco.

- A importância de definir claramente as fronteiras entre camadas: quando modelagem, persistência e interface estão em responsabilidades diferentes, alinhar interfaces técnicas cedo evita retrabalho.

4.2 Resultado alcançado

Ao final da Etapa 2, testei a integração completa da camada de banco com a interface JavaFX: a rotina PopularBanco gerou corretamente o arquivo pokedex.db com os 151 Pokémon, 91 golpes, 25 naturezas e 120 confrontos de efetividade; e a interface (AppPokedex) consumiu esses dados através dos repositórios sem escrever SQL, exibindo a Pokédex com busca, filtro por tipo e cálculo de fraquezas. A camada de dados cumpre o papel proposto de fonte única para as demais partes do sistema.

4.3 Conclusão

Ao longo das duas etapas, mantive a mesma responsabilidade — a camada de banco de dados — e a evolui de forma incremental: na Etapa 1, estabeleci a arquitetura (Database, repositórios, carga) e populei o banco a partir do CSV, complementando com dados canônicos que não constavam no arquivo original. Na Etapa 2, ampliei a camada com dois novos repositórios (Equipe e MembroTime) para atender à persistência do time do usuário, ataquei diretamente os pontos apontados no feedback (documentação Javadoc e commits com mensagens descritivas), e validei a integração com a interface. Considero a camada de dados concluída, estável e pronta para uso na entrega final do projeto.